Introdução à lógica de programação

1

Objetivos

Apresentar os conceitos elementares de lógica e sua aplicação no cotidiano. Definir algoritmo. Estabelecer uma relação entre lógica e algoritmos: a lógica de programação. Exemplificar a aplicação dos algoritmos utilizando situações do dia-a-dia. Comparar as principais formas de representação dos algoritmos.

- Introdução à lógica de programação
- Algoritmizando a lógica
- Conceitos e exemplos de algoritmos
- Noções de fluxos de controle

Noções de lógica

O que é lógica?

O uso corriqueiro da palavra lógica está normalmente relacionado à coerência e à racionalidade. Freqüentemente se associa lógica apenas à matemática, não se percebendo sua aplicabilidade e sua relação com as demais ciências.

Podemos relacionar a lógica com a 'correção do pensamento', pois uma de suas preocupações é determinar quais operações são válidas e quais não são, fazendo análises das formas e leis do pensamento. Como filosofia, ela procura saber por que pensamos assim não de outro jeito. Com arte ou técnica, ela nos ensina a usar corretamente as leis do pensamento.

Poderíamos dizer também que a lógica é a 'arte de bem pensar', que é a 'ciência das formas do pensamento'. Visto que a forma mais complexa do pensamento é o raciocínio, a lógica estuda a 'correção do raciocínio'. Podemos ainda dizer que a lógica tem em vista a 'ordem da razão'. Isso dá a entender que a nossa razão pode funcionar desordenadamente. Por isso, a lógica estuda e ensina a colocar 'ordem no pensamento'.

Exemplos

a. Todo mamífero é um animal.
Todo cavalo é um mamífero.
Portanto, todo cavalo é um animal.

b. Kaiton é país do planeta Stix.
Todos os Xinpins são de Kaiton.
Logo, todos os Xinpins são Stixianos.

Esses exemplos ilustram silogismos, que no estudo da Lógica Proposicional (ou Cálculo Sentencial) representam um argumento composto de duas premissas e uma conclusão; e está estabelecendo uma relação, que pode ser válida ou não. Esse é um dos objetivos da lógica, o estudo de técnicas de formalização, dedução e análise que permitam verificar a validade de argumentos. No caso dos exemplos, ambos são válidos.

Devemos ressaltar que, apesar da aparente coerência de um encadeamento lógico, ele pode ser válido ou não em sua estrutura. Nesse sentido, a lógica também objetiva a criação de uma representação mais formal, que se contrapõe à linguagem natural, que é suscetível a argumentações informais.

Existe lógica no dia-a-dia?

Sempre que pensamos, a lógica ou a ilógica necessariamente nos acompanham. Quando falamos ou escrevemos, estamos expressando nosso pensamento, logo, precisamos usar a lógica nessas atividades. Podemos perceber a importância da lógica em nossa vida, não só na teoria, como na prática, já que, quando queremos pensar, falar, escrever ou agir corretamente, precisamos colocar 'ordem no pensamento', isto é, utilizar lógica.

Exemplos

a. A gaveta está fechada.
A caneta está dentro da gaveta.
Precisamos primeiro abrir a gaveta para depois pegar a caneta.

b. Anacleto é mais velho que Felisberto.
Felisberto é mais velho que Marivaldo.
Portanto, Anacleto é mais velho que Marivaldo.

Mas e a lógica de programação?

Significa o uso correto das leis do pensamento, da 'ordem da razão' e de processos de raciocínio e simbolização formais na programação de computadores, objetivando a racionalidade e o desenvolvimento de técnicas que cooperem para a produção de soluções logicamente válidas e coerentes, que resolvam com qualidade os problemas que se deseja programar.

O raciocínio é algo abstrato, intangível. Os seres humanos têm a capacidade de expressá-lo através da palavra falada ou escrita, que por sua vez se baseia em um determinado idioma, que segue uma série de padrões (gramática). Um mesmo raciocínio pode ser expresso em

qualquer um dos inúmeros idiomas existentes, mas continuará representando o mesmo raciocínio, usando apenas outra convenção.

Algo similar ocorre com a Lógica de Programação, que pode ser concebida pela mente treinada e pode ser representada em qualquer uma das inúmeras linguagens de programação existentes. Essas, por sua vez, são muito atreladas a uma grande diversidade de detalhes computacionais, que pouco têm a ver com o raciocínio original. Para escapar dessa torre de Babel e, ao mesmo tempo, representar mais fielmente o raciocínio da Lógica de Programação, utilizamos os Algoritmos.

O QUE É UM ALGORITMO?

O objetivo principal do estudo da Lógica de Programação é a construção de algoritmos coerentes e válidos. Mas o que é um algoritmo?

Um **algoritmo** pode ser definido como uma seqüência de passos que visam a atingir um objetivo bem definido.

Na medida em que precisamos especificar uma seqüência de passos, é necessário utilizar ordem, ou seja, 'pensar com ordem', portanto precisamos utilizar lógica.

Apesar do nome pouco usual, algoritmos são comuns em nosso cotidiano, como, por exemplo, uma receita de bolo. Nela está descrita uma série de ingredientes necessários e uma seqüência de diversos passos (ações) que devem ser fielmente cumpridos para que se consiga fazer o alimento desejado, conforme se esperava antes do início das atividades (objetivo bem definido).

Quando elaboramos um algoritmo, devemos especificar ações claras e precisas, que a partir de um estado inicial, após um período de tempo finito, produzem um estado final previsível e bem definido. Isso significa que o algoritmo fixa um padrão de comportamento a ser seguido, uma norma de execução a ser trilhada, com vistas a alcançar, como resultado final, a solução de um problema, garantindo que sempre que executado, sob as mesmas condições, produza o mesmo resultado.

ALGORITMIZANDO A LÓGICA

POR QUE É IMPORTANTE CONSTRUIR UM ALGORITMO?

Um algoritmo tem por objetivo representar mais fielmente o raciocínio envolvido na Lógica de Programação e, dessa forma, permite-nos abstrair de uma série de detalhes computacionais, que podem ser acrescentados mais tarde. Assim, podemos focalizar nossa atenção naquilo que é importante: a lógica da construção de algoritmos.

Outra importância da construção dos algoritmos é que uma vez concebida uma solução algorítmica para um problema, esta pode ser traduzida para qualquer linguagem de programação e ser agregada das funcionalidades disponíveis nos diversos ambientes; costumamos denominar esse processo de codificação.

VAMOS A UM EXEMPLO?

Podemos escrever um primeiro algoritmo de exemplo, utilizando português coloquial, que descreva o comportamento na resolução de um determinada atividade, como, por

exemplo, a troca de uma lâmpada. Apesar de aparentemente óbvia demais, muitas vezes realizamos esse tipo de atividade inconscientemente, sem percebermos seus pequenos detalhes, que são as ações que nos levam a alcançar o objetivo proposto.

Vejamos esse primeiro algoritmo, descrito passo a passo:

ALGORITMO 1.1 Troca de lâmpada

- pegar uma escada;
- posicionar a escada embaixo da lâmpada;
- buscar uma lâmpada nova;
- subir na escada;
- retirar a lâmpada velha;
- colocar a lâmpada nova.

Involuntariamente, já seguimos uma determinada **seqüência** de ações que, representadas nesse algoritmo, fazem com que ele seja seguido naturalmente por qualquer pessoa, estabelecendo um padrão de comportamento, pois qualquer pessoa agiria da mesma maneira.

A **seqüenciação** é uma convenção com o objetivo de reger o fluxo de execução do algoritmo, determinando qual a primeira ação a ser executada e qual ação vem a seguir. Nesse caso, a seqüência é linear, de cima para baixo, assim como é a seqüência pela qual lemos um texto, de cima para baixo e da esquerda para a direita.

Reexaminando o algoritmo anterior, notamos que ele tem um objetivo bem definido: trocar uma lâmpada. Porém, e se a lâmpada não estivesse queimada? A execução das ações conduziria a uma troca, independentemente de a lâmpada estar ou não queimada, porque não foi prevista essa possibilidade em sua construção.

Para solucionar essa necessidade, podemos efetuar um teste, a fim de verificar se a lâmpada está ou não queimada. Uma solução para esse novo algoritmo seria:

ALGORITMO 1.2 Troca de lâmpada com teste

- pegar uma escada;
- posicionar a escada embaixo da lâmpada;
- buscar uma lâmpada nova;
- acionar o interruptor;
- se a lâmpada não acender, então
  - subir na escada;
  - retirar a lâmpada queimada;
  - colocar a lâmpada nova.

Agora estamos ligando algumas ações à condição `lâmpada não acender`, ou seja, se essa condição for verdadeira (lâmpada queimada) efetuaremos a troca da lâmpada, seguindo as próximas ações:

- subir na escada;
- retirar a lâmpada queimada;
- colocar a lâmpada nova.

Se a condição `lâmpada não acender` for falsa (a lâmpada está funcionando), as ações relativas à troca da lâmpada não serão executadas, e a lâmpada (que está em bom estado) não será trocada.

O que ocorreu nesse algoritmo foi a inclusão de um teste **seletivo**, através de uma condição que determina qual ou quais ações serão executadas (note que anteriormente, no **Algoritmo 1.1**, todas as ações eram executadas), dependendo da inspeção da condição resultar em verdadeiro ou falso.

Esse algoritmo está correto, uma vez que atinge seu objetivo, porém, pode ser melhorado, uma vez que buscamos uma escada e uma lâmpada sem saber se serão necessárias. Mudemos então o teste condicional `se a lâmpada não acender` para o início da seqüência de ações:

ALGORITMO 1.3 Troca de lâmpada com teste no início

```
• acionar o interruptor;
• se a lâmpada não acender, então
   • pegar uma escada;
   • posicionar a escada embaixo da lâmpada;
   • buscar uma lâmpada nova;
   • acionar o interruptor;
   • subir na escada;
   • retirar a lâmpada queimada;
   • colocar a lâmpada nova.
```

Observe que, agora, a ação `acionar o interruptor` é a primeira do algoritmo e a condição `lâmpada não acender` já é avaliada. Nesse caso, `pegar uma escada` até `colocar a lâmpada nova` dependem de a lâmpada estar efetivamente queimada. Há muitas formas de resolver um problema, afinal cada pessoa pensa e age de maneira diferente, cada indivíduo tem uma heurística própria. Isso significa que, para esse mesmo problema de trocar lâmpadas, poderíamos ter diversas soluções diferentes e corretas (se atingissem o resultado desejado de efetuar a troca), portanto, o bom senso e a prática de lógica de programação é que indicarão qual a solução mais adequada, que com menos esforço e maior objetividade produzirá o resultado almejado.

A solução apresentada no **Algoritmo 1.3** é aparentemente adequada, porém não prevê a possibilidade de a lâmpada nova não funcionar e, portanto, não atingir o objetivo nessa situação específica. Podemos fazer um refinamento, uma melhoria no algoritmo, de tal modo que se troque a lâmpada diversas vezes, se necessário, até que funcione. Uma solução seria:

ALGORITMO 1.4 Troca de lâmpada com teste e repetição indefinida

```
• acionar o interruptor;
• se a lâmpada não acender, então
   • pegar uma escada;
   • posicionar a escada embaixo da lâmpada;
   • buscar uma lâmpada nova;
```

(Continua)

- acionar o interruptor;
- subir na escada;
- retirar a lâmpada queimada;
- colocar a lâmpada nova;
- se a lâmpada não acender, então
  - retirar a lâmpada queimada;
  - colocar outra lâmpada nova;
  - se a lâmpada não acender, então
    - retirar a lâmpada queimada;
    - colocar outra lâmpada nova;
    - se a lâmpada não acender, então
      - retirar a lâmpada queimada;
      - colocar outra lâmpada nova;

        .
        .
        .

Até quando?

Notamos que o **Algoritmo 1.4** não está terminado, falta especificar até quando será feito o teste da lâmpada. As ações cessarão quando conseguirmos colocar uma lâmpada que acenda; caso contrário, ficaremos testando indefinidamente (note que o interruptor continua acionado!). Essa solução está mais próxima do objetivo, pois garante que a lâmpada acenda novamente, ou melhor, que seja trocada com êxito, porém, temos o problema de não saber o número exato de testes das lâmpadas.

Observemos que o teste da lâmpada nova é efetuado por um mesmo conjunto de ações:

- se a lâmpada não acender, então
  - retirar a lâmpada queimada;
  - colocar uma lâmpada nova.

Portanto, em vez de reescrevermos várias vezes esse conjunto de ações, podemos alterar o fluxo seqüencial de execução de forma que, após executada a ação `colocar outra lâmpada nova`, voltemos a executar o teste `se a lâmpada não acender`, fazendo com que essas ações sejam executadas o número de vezes necessário sem termos de reescrevê-las.

Precisamos, então, expressar essa repetição da ação sem repetir o texto que representa a ação, assim como determinar um limite para tal repetição, com o objetivo de garantir uma **condição de parada**, ou seja, que seja cessada a atividade de testar a lâmpada nova quando ela já estiver acesa. Uma solução seria:

- enquanto a lâmpada não acender, faça
  - retirar a lâmpada queimada;
  - colocar uma lâmpada nova.

A condição `lâmpada não acender` permaneceu e estabelecemos um fluxo repetitivo que será finalizado assim que a condição de parada for falsa, ou seja, assim que a lâmpada acender. Percebemos que o número de repetições é **indefinido**, porém é **finito**, e que depende

apenas da condição estabelecida, o que leva a repetir as ações até alcançar o objetivo: trocar a lâmpada queimada por uma que funcione. O novo algoritmo ficaria:

ALGORITMO 1.5 Troca de lâmpada com teste e condição de parada

- acionar o interruptor;
- se a lâmpada não acender, então
  - pegar uma escada;
  - posicionar a escada embaixo da lâmpada;
  - buscar uma lâmpada nova;
  - acionar o interruptor;
  - subir na escada;
  - retirar a lâmpada queimada;
  - colocar uma lâmpada nova;
  - enquanto a lâmpada não acender, faça
    - retirar a lâmpada queimada;
    - colocar uma lâmpada nova;

Até agora estamos efetuando a troca de uma única lâmpada, na verdade estamos testando um soquete (acionado por um interruptor), trocando tantas lâmpadas quantas forem necessárias para assegurar que o conjunto funcione. O que faríamos se tivéssemos mais soquetes a testar, por exemplo, dez soquetes?

A solução aparentemente mais óbvia seria repetir o algoritmo de uma única lâmpada para os dez soquetes existentes, ficando algo como:

ALGORITMO 1.6 Troca de lâmpada com teste para 10 soquetes

- acionar o interruptor do **primeiro** soquete;
- se a lâmpada não acender, então
  - pegar uma escada;
  - posicionar a escada embaixo da lâmpada;
  - buscar uma lâmpada nova;
  - acionar o interruptor;
  - subir na escada;
  - retirar a lâmpada queimada;
  - colocar uma lâmpada nova;
  - enquanto a lâmpada não acender, faça
    - retirar a lâmpada queimada;
    - colocar uma lâmpada nova;
- acionar o interruptor do **segundo** soquete;
- se a lâmpada não acender, então
  - pegar uma escada;
  - posicionar a escada embaixo da lâmpada;

.
.
.

- acionar o interruptor do **terceiro** soquete;
- se a lâmpada não acender, então

(Continua)

```
                .
                .
                .
• acionar o interruptor do quarto soquete;
                .
                .
                .
• acionar o interruptor do décimo soquete;
                .
                .
                .
```

Observamos que o **Algoritmo 1.6** é apenas um conjunto de dez repetições do **Algoritmo 1.5**, uma vez para cada soquete, havendo a repetição de um mesmo conjunto de ações por um número definido de vezes: dez. Como o conjunto de ações que foram repetidas é exatamente igual, poderíamos alterar o fluxo seqüencial de execução de modo a fazer com que ele voltasse a executar o conjunto de ações relativas a um único soquete (**Algoritmo 1.5**) tantas vezes quantas fossem desejadas. Uma solução para dez soquetes seria:

ALGORITMO 1.7 Troca de lâmpada com teste para 10 soquetes com repetição

```
• ir até o interruptor do primeiro soquete;
• enquanto a quantidade de soquetes testados for menor que dez, faça
    • acionar o interruptor;
    • se a lâmpada não acender, então
        • pegar uma escada;
        • posicionar a escada embaixo da lâmpada;
        • buscar uma lâmpada nova;
        • acionar o interruptor;
        • subir na escada;
        • retirar a lâmpada queimada;
        • colocar uma lâmpada nova;
        • enquanto a lâmpada não acender, faça
            • retirar a lâmpada queimada;
            • colocar uma lâmpada nova;
• ir até o interruptor do próximo soquete;
```

Quando a condição `quantidade de soquetes testados for menor que dez` for verdadeira, as ações responsáveis pela troca ou não de um único soquete serão executadas. Caso a condição de parada seja falsa, ou seja, todos os dez soquetes já tiverem sido testados, nada mais será executado.

Todo o exemplo foi desenvolvido a partir do problema de descrevermos os passos necessários para efetuar a troca de uma lâmpada, ou seja, construir um **algoritmo** para esse fim.

Inicialmente, tínhamos um pequeno conjunto de ações que deveriam ser executadas, todas passo a passo, uma após a outra, compondo uma ordem seqüencial de execução, a estrutura **seqüencial**.

Notamos que nem sempre todas as ações previstas deveriam ser executadas. Tal circunstância sugeriu que um determinado conjunto de ações fosse evitado, selecionando conforme o resultado de uma determinada condição. Construímos, assim, uma **estrutura seletiva** através de um **teste condicional** que permitia ou não que o fluxo de execução passasse por um determinado conjunto de ações.

Quando deparamos com a inviabilidade da aplicação da estrutura de seleção para a verificação do êxito na troca da lâmpada, precisamos repetir um mesmo trecho do algoritmo, o que foi realizado alterando-se o fluxo de execução de modo que ele passasse pelo mesmo trecho diversas vezes, enquanto a condição não fosse satisfeita; agimos de forma semelhante na situação de trocar dez lâmpadas, construindo uma **estrutura de repetição**.

Devemos ressaltar que qualquer pessoa, fundamentada na própria experiência, seria capaz de resolver o problema na prática, envolvendo inclusive circunstâncias inusitadas que pudessem surgir. Contudo, um programa de computador tradicional não tem conhecimento prévio nem adquire experiências, o que implica que devemos determinar em detalhes todas as ações que ele deve executar, prevendo todos os obstáculos e a forma de transpô-los, isto é, descrever uma seqüência finita de passos que garantam a solução do problema. Tal atividade é realizada pelos **programadores**, que podemos chamar de **construtores de algoritmos**.

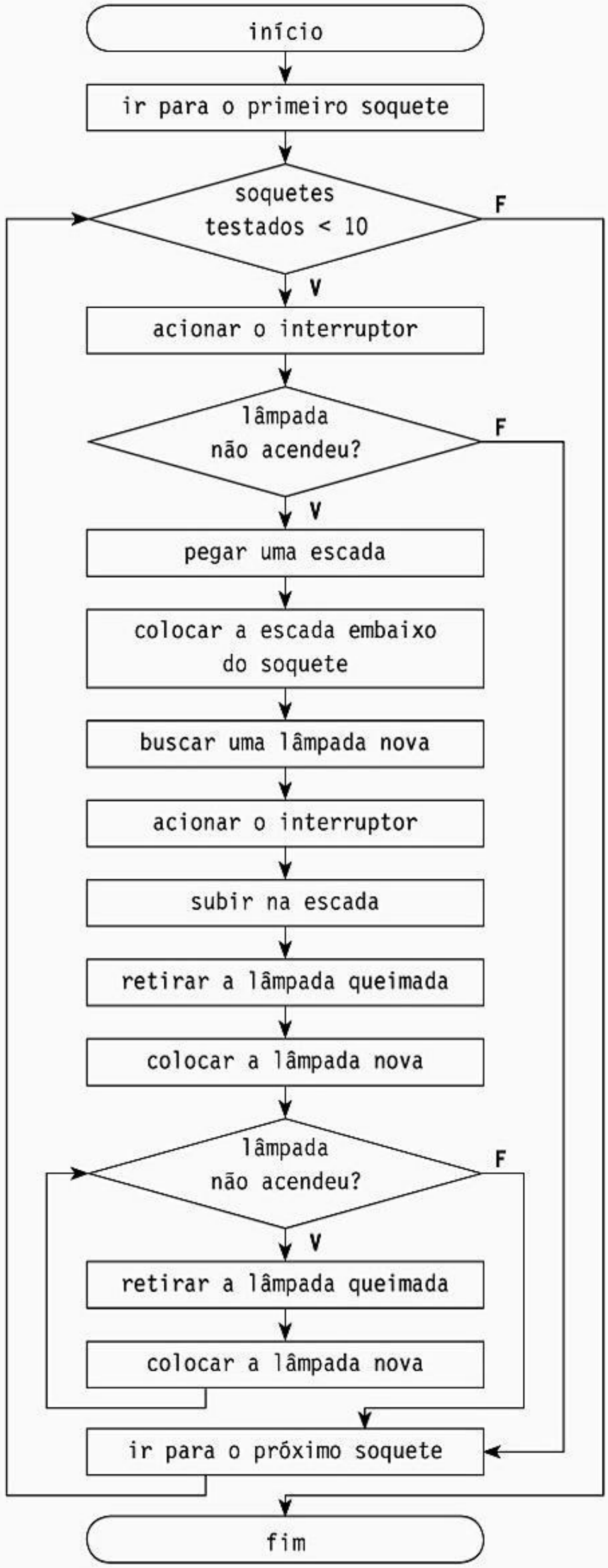
De que maneira representaremos o algoritmo?

Convém enfatizar mais uma vez que um algoritmo é uma linha de raciocínio, que pode ser descrito de diversas maneiras, de forma gráfica ou textual.

Os algoritmos representados até o momento estavam em forma textual, usando português coloquial.

As formas gráficas são mais puras por serem mais fiéis ao raciocínio original, substituindo um grande número de palavras por convenções de desenhos. Para fins de ilustração mostraremos como ficaria o **Algoritmo 1.7** representado graficamente em um fluxograma tradicional (**Algoritmo 1.8**)

ALGORITMO 1.8 Fluxograma



Para representar textualmente algoritmos usaremos o português, como já vínhamos utilizando. Todavia, não poderíamos utilizar toda a riqueza gramatical de nossa língua pátria. Discorremos sobre pelo menos um bom e claro motivo: a ambigüidade.

Vejamos a seguinte frase: "O pregador foi grampeado durante o conserto". Esse exemplo, quando falado, pode ter até oito sentidos diferentes, uma vez que pregador pode ser um religioso que prega a palavra de Deus ou um prendedor de roupas; grampeado pode se tratar de uma escuta telefônica ou do grampo que une folhas de papel; conserto, quando pronunciado, pode se tratar de uma apresentação musical ou da manutenção em algum objeto. Conseguiríamos até distinguir qual dos oito sentidos diferentes se aplicaria, caso avaliássemos a sentença dentro de seu contexto. Entretanto, o computador é desprovido do raciocínio necessário para interpretar a frase.

Para evitar esse e outros problemas, utilizaremos um conjunto de regras que visam restringir e estruturar o uso do português na representação dos algoritmos e que, intencionalmente, se aproximam da maneira pela qual o fazem as linguagens de programação reais (como C e Pascal), com a finalidade de facilitar a futura codificação dos algoritmos.

Até então, já sabemos o que é um algoritmo e já estamos aptos a escrever algoritmos (e resolver os exercícios do capítulo). A partir do próximo capítulo agregaremos alguns conceitos e estabeleceremos o conjunto de regras de escrita de nosso português estruturado.

Exercícios de fixação 1

1.1 Três senhoras – dona Branca, dona Rosa e dona Violeta – passeavam pelo parque quando dona Rosa disse:

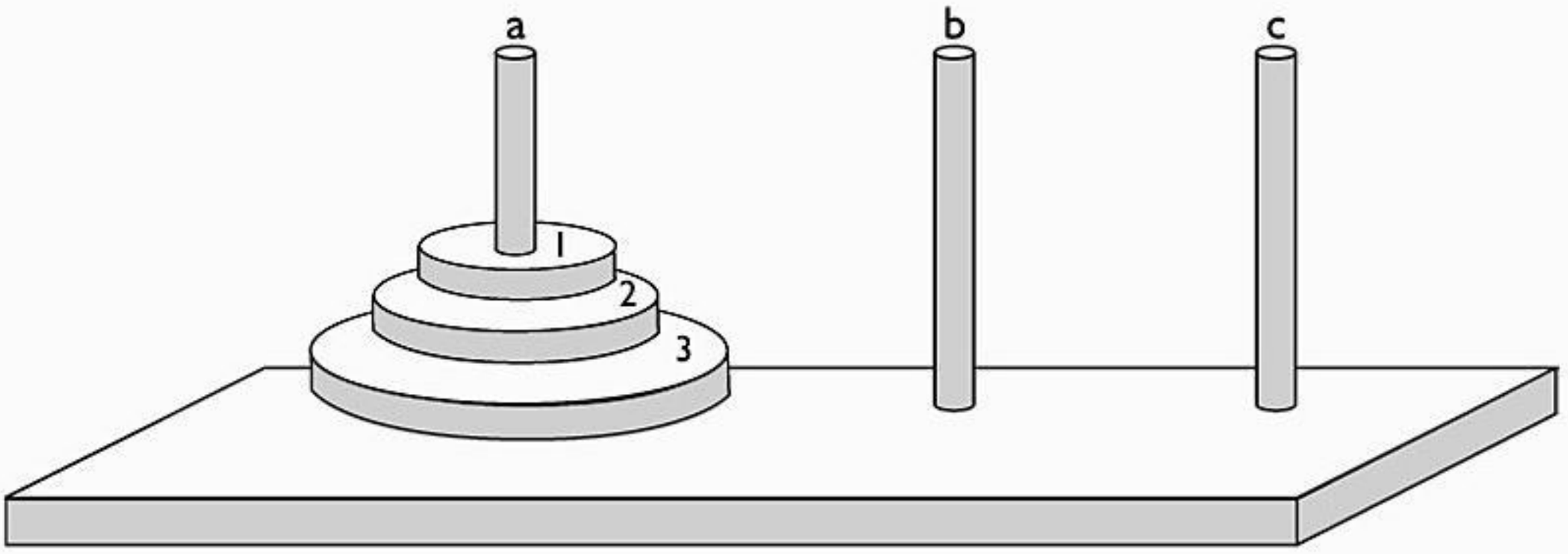
– Não é curioso que estejamos usando vestidos de cores branca, rosa e violeta, embora nenhuma de nós esteja usando um vestido de cor igual ao seu próprio nome?

– Uma simples coincidência – respondeu a senhora com o vestido violeta.

Qual a cor do vestido de cada senhora ?

1.2 Um homem precisa atravessar um rio com um barco que possui capacidade apenas para carregar ele mesmo e mais uma de suas três cargas, que são: um lobo, um bode e um maço de alfafa. O que o homem deve fazer para conseguir atravessar o rio sem perder suas cargas? Escreva um algoritmo mostrando a resposta, ou seja, indicando todas as ações necessárias para efetuar uma travessia segura.

1.3 Elabore um algoritmo que mova três discos de uma Torre de Hanói, que consiste em três hastes ($a - b - c$), uma das quais serve de suporte para três discos de tamanhos diferentes ($1 - 2 - 3$), os menores sobre os maiores. Pode-se mover um disco de cada vez para qualquer haste, contanto que nunca seja colocado um disco maior sobre um menor. O objetivo é transferir os três discos para outra haste.



1.4 Três jesuítas e três canibais precisam atravessar um rio; para tal, dispõem de um barco com capacidade para duas pessoas. Por medida de segurança, não se deve permitir que em alguma margem a quantidade de jesuítas seja inferior à de canibais. Qual a solução para efetuar a travessia com segurança? Elabore um algoritmo mostrando a resposta, indicando as ações que concretizam a solução deste problema.

Anexo

Resolução dos exercícios de fixação

Exercício 1.1 (página 12)

Se a senhora com o vestido violeta respondeu a dona Rosa, então ela não é a própria dona Rosa. Além disso, como ela não tem o vestido da mesma cor de seu nome, ela também não é a dona Violeta. Logo, é dona Branca que está com o vestido violeta. Dona Rosa não está usando o vestido rosa nem o violeta, portanto só pode estar usando o branco. Conseqüentemente, dona Violeta veste o vestido rosa.

Exercício 1.2 (página 12)

- levar o bode para o outro lado do rio;
- voltar sem carga nenhuma;
- levar o lobo para o outro lado do rio;
- voltar com o bode;
- levar a alfafa para o outro lado do rio;
- voltar sem carga nenhuma;
- levar o bode para o outro lado do rio.

Exercício 1.3 (página 12)

- mover o disco 1 da torre A para a torre B;
- mover o disco 2 da torre A para a torre C;
- mover o disco 1 da torre B para a torre C;

- mover o disco 3 da torre A para a torre B;
- mover o disco 1 da torre C para a torre A;
- mover o disco 2 da torre C para a torre B;
- mover o disco 1 da torre A para a torre B.

Exercício 1.4 (página 12)

- atravessar um jesuíta e um canibal para a margem B;
- voltar o jesuíta para a margem A;
- atravessar dois canibais para a margem B;
- voltar um canibal para a margem A;
- atravessar dois jesuítas para a margem B;
- voltar um jesuíta e um canibal para a margem A;
- atravessar dois jesuítas para a margem B;
- voltar um canibal para a margem A;
- atravessar dois canibais para a margem B;
- voltar um canibal para a margem A;
- atravessar dois canibais para a margem B.